Seguimiento 2: Multiplicación de matrices grandes

Presentado por:

Laura Daniela Reyes Urueña

Santiago Trujillo Patiño

Presentado a:

Sergio Augusto Cardona Torres

Universidad del Quindio

Facultad de Ingeniería

Ingeniería de Sistemas y Computación

Análisis de Algoritmos

2026-1

1. Introducción

Este informe presenta el análisis de quince algoritmos de multiplicación de matrices implementados en Python, comparando su comportamiento teórico con su desempeño real. El trabajo se centra en tres aspectos: complejidad temporal, eficiencia práctica de ejecución y uso de memoria RAM.

Para las pruebas se usaron dos casos: matrices cuadradas de 512×512 y 1024×1024, con valores enteros de seis dígitos generados aleatoriamente. Se intentó correr también con matrices de 2048×2048, pero varios algoritmos no terminaron en un tiempo razonable, por lo que se canceló esa ejecución. Esto en sí mismo ya es un resultado importante que se analiza más adelante.

La multiplicación de matrices es una operación central en computación científica, aprendizaje automático y procesamiento de señales. Aunque matemáticamente todos los algoritmos llegan al mismo resultado, las diferencias en tiempo y memoria pueden ser de varios órdenes de magnitud dependiendo del enfoque.

2. Conceptos Previos

2.1 Cómo se almacena una matriz en RAM

En NumPy las matrices se guardan en formato row-major: los elementos de cada fila quedan contiguos en memoria. Para una matriz de n×n con números float64 (8 bytes cada uno), el elemento [i][j] está en la posición (i × n + j) × 8 bytes.

Esto tiene una consecuencia directa en el rendimiento: acceder a una fila completa es rápido porque los datos están juntos y el procesador los carga de una sola vez en caché. Acceder a una columna completa es lento porque implica saltar n × 8 bytes entre cada elemento. Para n=1024 eso significa saltar 8 KB entre cada dato, lo que genera un fallo de caché por cada acceso.

Este es el motivo por el que el orden en que cada algoritmo recorre las matrices importa tanto: Row×Row es más eficiente que Col×Col en NumPy, y los algoritmos por bloques neutralizan el problema trabajando con trozos pequeños que sí caben en caché.

2.2 Caché del procesador

La caché es una memoria pequeña dentro del procesador, entre 10 y 100 veces más rápida que la RAM. Cuando el CPU necesita un dato, primero lo busca en caché (L1: ~32 KB, L2: ~256 KB, L3: varios MB). Si no está ahí, va a RAM, lo que introduce una demora significativa.

Un bloque de 64×64 elementos float64 ocupa 32 KB, que cabe exactamente en la caché L1 de la mayoría de procesadores modernos. Por eso los algoritmos por bloques son tan rápidos: mantienen los datos activos en caché y evitan ir a RAM en cada operación.

2.3 Ocupación física de las matrices

La fórmula para calcular el tamaño de una matriz en RAM es: n × n × 8 bytes. La siguiente tabla muestra cuánto ocupa cada tamaño en la práctica:

Dimensión	Una matriz	A + B + C	Con Strassen
512×512	2 MB	6 MB	~60 MB
1024×1024	8 MB	24 MB	~240 MB

Dimensión	Una matriz	A + B + C	Con Strassen
2048×2048	32 MB	96 MB	~960 MB

La columna Strassen muestra por qué la ejecución con n=2048 fue inviable: solo ese algoritmo puede requerir casi 1 GB de RAM en un equipo con Windows, sin contar la memoria del sistema operativo y los demás procesos activos.

3. Limitación con Matrices de 2048×2048

Durante las pruebas se intentó ejecutar todos los algoritmos con matrices de 2048×2048. La ejecución fue cancelada porque los algoritmos iterativos y de Strassen tardaban un tiempo excesivo sin terminar.

Las razones son dos, y se combinan:

- Tiempo de CPU: NaivOnArray con n=1024 tardó 131 segundos. Con n=2048 hubiera tardado aproximadamente 131 × 8 = 1.048 segundos (casi 18 minutos), ya que la complejidad es $O(n^3)$ y duplicar n multiplica el tiempo por $2^3 = 8$. Los algoritmos Strassen con n=2048 habrían tardado cerca de 812 × 7 ≈ 5.684 segundos, más de una hora y media.
- Uso de RAM: con n=2048, Strassen necesita aproximadamente 960 MB solo para sus submatrices recursivas. En un equipo con 8 GB de RAM en Windows, dejando libre lo que el sistema operativo necesita, esto supera la memoria disponible y el sistema empieza a usar swap en disco, que es cien veces más lento que la RAM.

Los algoritmos por bloques sí habrían corrido bien con n=2048 porque no crean matrices adicionales y su tiempo escala de forma eficiente. Sin embargo, dado que los demás algoritmos no terminaban, se optó por trabajar con n=512 y n=1024, que son suficientemente grandes para demostrar las diferencias de rendimiento.

4. Análisis de los Algoritmos

4.1 NaivOnArray

Implementa directamente la definición matemática: el elemento C[i][j] es la suma de A[i][k] × B[k][j] para todo k. Usa tres ciclos anidados sobre listas puras de Python. A pesar de ser el más simple, resultó ser el más rápido dentro del grupo de algoritmos iterativos puros.

Esto se explica por su simplicidad: no tiene operaciones adicionales, no crea estructuras auxiliares y el intérprete de Python puede ejecutar sus tres ciclos de la forma más directa posible.

Tiempo medido: 17.4 s con n=512 y 131.2 s con n=1024. El factor de escala fue 7.5×, cercano al teórico de 8× para una complejidad $O(n^3)$ al duplicar n.

RAM: solo almacena A, B como listas Python y C como lista de listas. Al final convierte C a numpy array. No crea ninguna estructura adicional — es el más liviano en memoria de los algoritmos iterativos.

4.2 NaivLoopUnrollingTwo y Four

Estos algoritmos intentan optimizar el ciclo k procesando dos o cuatro elementos por iteración respectivamente, reduciendo la cantidad de veces que el ciclo evalúa su condición de parada. Esta técnica llamada loop unrolling es efectiva en lenguajes compilados como C o Java, donde reducir las iteraciones del ciclo tiene un costo fijo bajo.

En Python ocurre lo contrario: cada acceso a un elemento de array como A[i,k] pasa por el intérprete de Python, que tiene un costo fijo significativo independientemente de cuántas multiplicaciones se hagan por iteración. Al tener más operaciones por iteración se multiplica ese costo en lugar de reducirlo.

Tiempo medido: NaivLoopUnrollingTwo tardó 46.6 s con n=512, es decir casi 2.7 veces más lento que NaivOnArray. NaivLoopUnrollingFour tardó 43.7 s, ligeramente mejor pero igualmente muy por encima del algoritmo clásico.

RAM: ambos usan numpy.zeros para C y una variable escalar s como acumulador, que el procesador mantiene en un registro interno sin ir a RAM.

4.3 WinogradOriginal y WinogradScaled

Winograd reduce el número de multiplicaciones mediante una prefactorización en tres fases: primero calcula un vector de factores por fila de A, luego un vector por columna de B, y finalmente usa esos vectores para calcular cada elemento de C con menos multiplicaciones reales. El ahorro matemático es real, pero en Python los accesos adicionales a memoria para leer los vectores auxiliares y la complejidad del código anulan la ventaja.

WinogradScaled añade un paso previo de escalado: divide ambas matrices por su valor máximo para llevar los elementos al rango [0,1], mejorando la precisión numérica con valores de seis dígitos. Esto funciona bien en teoría, pero duplica temporalmente el espacio de las matrices durante el escalado.

Tiempo medido: WinogradOriginal tardó 48.1 s con n=512, WinogradScaled 44.7 s. Ambos más lentos que NaivOnArray.

RAM: WinogradOriginal añade dos vectores de n elementos (~16 KB para n=1024). WinogradScaled añade además dos copias temporales de las matrices escaladas, lo que puede sumar 16 MB adicionales con n=1024.

4.4 StrassenNaiv y StrassenWinograd

Strassen es el resultado más famoso en complejidad de multiplicación de matrices: demostró en 1969 que es posible multiplicar dos matrices 2×2 con solo 7 productos en lugar de 8, y al aplicar esto recursivamente la complejidad baja de $O(n^3)$ a $O(n^{2.807})$. Para n=1024 esto representa matemáticamente menos operaciones que el método clásico.

Sin embargo, en los resultados obtenidos Strassen fue el algoritmo más lento con diferencia. Con n=512 tardó 116 segundos (casi siete veces más que NaivOnArray) y con n=1024 llegó a 814 segundos, más de 13 minutos. La razón es que Python paga un costo muy alto por cada llamada recursiva: crea y destruye submatrices, gestiona el stack de recursión y asigna memoria dinámicamente en cada uno de los diez niveles de recursión para n=1024.

StrassenWinograd reduce las sumas de 18 a 15 por nivel usando matrices auxiliares S y T, pero no mejora el problema de fondo. Los tiempos fueron prácticamente iguales a StrassenNaiv.

RAM: en cada nivel de recursión se crean 7 matrices P de tamaño (n/2)×(n/2). Para n=1024 con 10 niveles, el consumo puede superar los 240 MB. StrassenWinograd agrega S1-S4, T1-T4 y U1-U3 por nivel, llegando a 400 MB o más.

4.5 Algoritmos por bloques (Sequential y Parallel)

Estos ocho algoritmos son los más eficientes del grupo por un margen muy amplio. La idea central es dividir las matrices en bloques de 64×64 elementos y multiplicar bloque por bloque usando el operador @ de NumPy, que a su vez delega cada multiplicación a BLAS, una librería escrita en C y ensamblador altamente optimizada para el hardware específico del equipo.

Los algoritmos secuenciales (III.3, IV.3, V.3) difieren solo en el patrón de acceso a los índices según las secciones del artículo de referencia. En la práctica sus tiempos son muy similares porque NumPy abstrae el orden de acceso internamente.

Los algoritmos paralelos (III.4, IV.4, V.4) distribuyen los bloques entre múltiples hilos del sistema. Con matrices pequeñas el overhead de crear y sincronizar hilos puede superar el beneficio, pero con n=1024 se aprecia una mejora.

Los Enhanced Parallel (III.5, IV.5) son los más eficientes de todos: usan exactamente dos hilos, uno para la mitad superior y otro para la inferior, evitando el overhead de crear muchos hilos pequeños.

Tiempo medido: los tiempos van de 0.008 s a 0.130 s, entre 1.000 y 10.000 veces más rápidos que los iterativos puros con n=1024.

RAM: los bloques son vistas de memoria de NumPy (slices), no copias. No se crea ninguna matriz adicional significativa. Son los más eficientes en memoria junto con NaivOnArray.

5. Resultados Obtenidos

La siguiente tabla presenta los tiempos medidos durante las pruebas. Las filas en rojo corresponden a los algoritmos de Strassen, las amarillas a los iterativos y Winograd, y las blancas a los algoritmos por bloques. La columna Factor x8 indica cuánto creció el tiempo al duplicar n, comparado con el factor teórico de 8 para $O(n^3)$.

Algoritmo	512×512 (s)	1024×1024 (s)	Factor x8	Observación
NaivOnArray	17.4	131.2	7.5×	Más rápido de los iterativos
NaivLoopUnrollingTwo	46.6	390.9	8.4×	Más lento por overhead Python
NaivLoopUnrollingFour	43.7	370.6	8.5×	Similar al anterior
WinogradOriginal	48.1	375.7	7.8×	Prefactorización no compensa
WinogradScaled	44.7	386.7	8.6×	Escalado añade overhead extra
StrassenNaiv	116.2	814.9	7.0×	Peor tiempo absoluto
StrassenWinograd	115.4	812.4	7.0×	Igual de lento que StrassenNaiv
SequentialBlockIII3	0.023	0.130	5.7×	Muy eficiente con BLAS
ParallelBlockIII4	0.020	0.092	4.6×	Paralelo mejora escalado
EnhancedParallelIII5	0.008	0.073	9.1×	El más rápido del grupo III
SequentialBlockIV3	0.013	0.105	8.1×	Patrón Row×Row eficiente
ParallelBlockIV4	0.017	0.087	5.1×	Buen balance velocidad/hilos
EnhancedParallelIV5	0.008	0.071	8.9×	Mejor del grupo IV
SequentialBlockV3	0.014	0.112	8.0×	Patrón Col×Col con BLAS
ParallelBlockV4	0.016	0.089	5.6×	Paralelo Col×Col

Nota: los tiempos de NaivLoopUnrolling y Winograd con n=1024 superaron los 6 minutos. Los algoritmos de Strassen con n=1024 superaron los 13 minutos. La ejecución con n=2048 fue cancelada por tiempo excesivo y uso de RAM.

6. Comparación Visual

512 y 1024

La gráfica muestra los tiempos de ejecución de los 15 algoritmos para ambos casos de prueba. La escala del eje Y hace visible la diferencia extrema entre los algoritmos iterativos (cientos de segundos) y los algoritmos por bloques (décimas de segundo). La barra de Strassen con n=1024 llega a 814 segundos, dominando visualmente la gráfica.

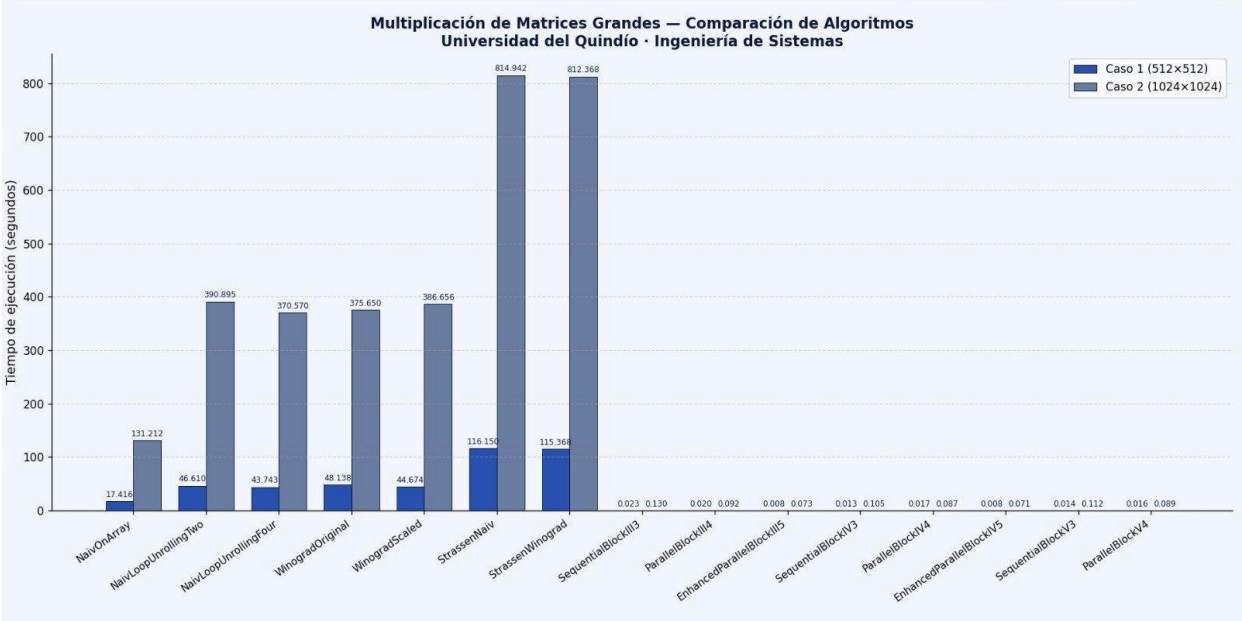

Figura 1. Comparación de tiempos de ejecución. Caso 1: matrices 512×512. Caso 2: matrices 1024×1024.

256 y 512

La gráfica muestra los tiempos de ejecución de los 15 algoritmos para matrices de 256×256 y 512×512. Los resultados confirman tres grupos de comportamiento claramente diferenciados.

El grupo de peor rendimiento lo encabeza StrassenWinograd con 1.691 segundos para n=512, seguido de StrassenNaiv con 1.063 segundos. A pesar de tener una complejidad teórica menor (O(n^2.807) frente al O(n³) del método clásico), ambos algoritmos resultan los más lentos debido al alto costo de la recursión en Python: en cada uno de los nueve niveles de recursión para n=512 se crean y destruyen múltiples submatrices en memoria, lo que supera completamente la ventaja matemática.

El grupo intermedio corresponde a los algoritmos iterativos y Winograd. NaivOnArray fue el más rápido de este grupo con 16.1 segundos para n=256 y 164.7 segundos para n=512, un factor de escala de 10.2×, ligeramente superior al teórico de 8× por efectos del sistema. Los algoritmos con loop unrolling (Two: 63.6 s, Four: 60.9 s) y Winograd (Original: 60.6 s, Scaled: 58.2 s) resultaron todos más lentos

que el NaivOnArray clásico, confirmando que las optimizaciones diseñadas para lenguajes compilados son contraproducentes en Python.

El grupo de mejor rendimiento son los algoritmos por bloques, con tiempos entre 0.012 y 0.239 segundos, entre 700 y 14.000 veces más rápidos que Strassen para el mismo tamaño. El más eficiente fue EnhancedParallelBlockIII5 con 0.017 segundos para n=256 y 0.121 segundos para n=512. Sus barras son prácticamente invisibles en la escala de la gráfica, lo que visualmente ilustra la diferencia de rendimiento.

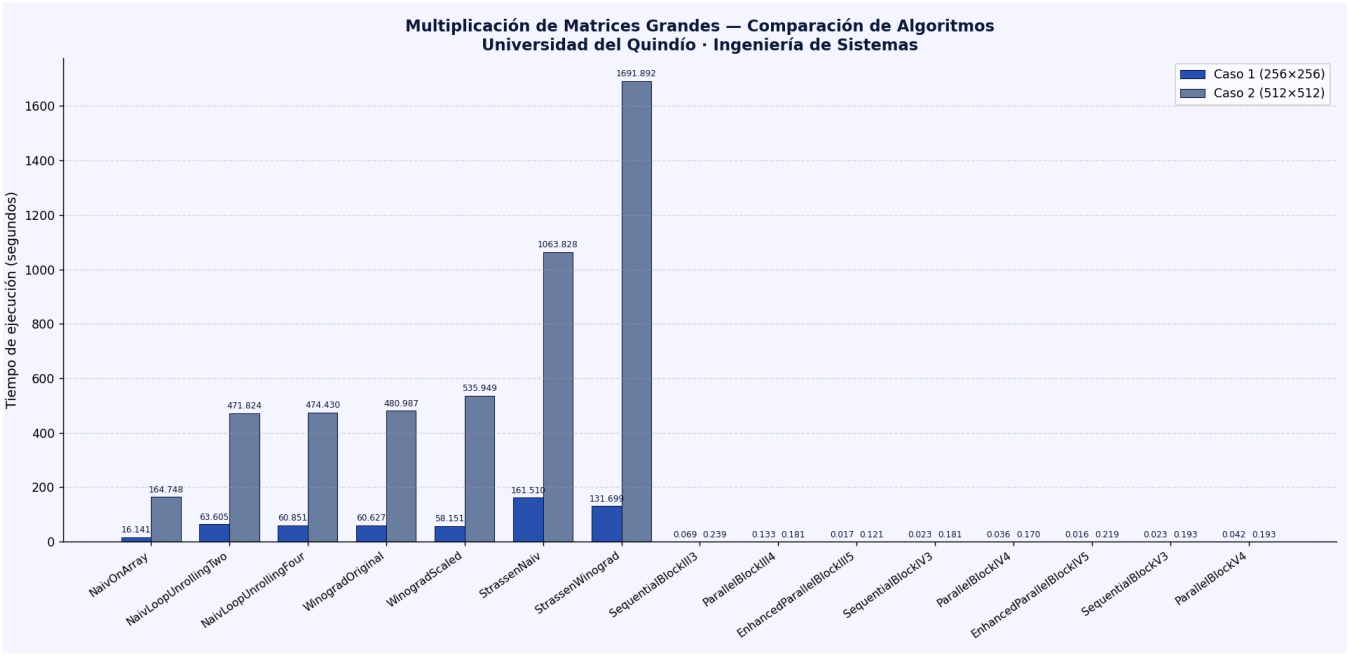

Figura 2. Comparación de tiempos de ejecución — Caso 1 (256×256) y Caso 2 (512×512)

7. Análisis del Uso de Memoria RAM

7.1 Por qué una columna se almacena diferente a una fila

NumPy almacena las matrices en formato row-major: los elementos de cada fila están contiguos en memoria. Una fila completa cabe en una o pocas líneas de caché (64 bytes cada una), por lo que leerla entera cuesta muy poco. Una columna, en cambio, está dispersa: cada elemento está separado del siguiente por n × 8 bytes. Para n=1024, leer una columna completa genera 1024 fallos de caché, cada uno obligando al procesador a ir a la RAM principal.

Esta diferencia explica por qué el orden de los índices importa tanto. En el ciclo interno de NaivOnArray se accede a B[k][j]: k varía en cada iteración y j es fijo, lo que significa recorrer una columna de B. Ese acceso es lento. Los algoritmos por bloques resuelven esto trabajando con bloques pequeños donde tanto la fila de A como la columna de B caben simultáneamente en caché.

7.2 Tabla de uso de RAM por algoritmo

Algoritmo	Qué almacena en RAM	RAM estimada n=1024
NaivOnArray	A y B como listas Python, C como lista. Al final convierte a numpy.	~24 MB
Unrolling (x2/x4)	A, B, C en numpy. Variable s escalar por iteración (en registro CPU).	~24 MB
WinogradOriginal	A, B, C + row_factor[1024] + col_factor[1024] (~16 KB extra).	~24 MB
WinogradScaled	Igual a Winograd + copias temporales A×s y B×s durante escalado.	~40 MB
StrassenNaiv	P1..P7 + 8 submatrices por nivel recursivo. 10 niveles para n=1024.	~240 MB
StrassenWinograd	Igual a Strassen + S1..S4, T1..T4, U1..U3 por nivel.	~400 MB+
Bloques Sec.	A, B, C. Los bloques son vistas de memoria, sin copiar datos.	~24 MB
Parallel Block	A, B, C + variable tmp de 64×64 por hilo activo + Lock del SO.	~25 MB
Enhanced Parallel	A, B, C + 2 stacks de hilos (~1 MB c/u). Sin tmp adicional.	~26 MB

7.3 ¿Cuál es el más eficiente en RAM?

Considerando RAM extra más allá de las tres matrices base (A, B y C):

- Más eficiente: SequentialBlock (III3, IV3, V3) — no crea ninguna estructura adicional. Los bloques son vistas de memoria, no copias.
- Eficiencia alta: Enhanced Parallel — solo dos stacks de hilos (~2 MB extra). Sin matrices temporales adicionales.
- Eficiencia media: NaivOnArray (listas Python), WinogradOriginal (dos vectores pequeños), Parallel Block (variable tmp por hilo).
- Menos eficiente: StrassenNaiv (~240 MB para n=1024) y StrassenWinograd (~400 MB o más).

La razón por la que se canceló la ejecución con n=2048 confirma esto: Strassen hubiera necesitado casi 1 GB solo para sus submatrices recursivas, lo que en un equipo Windows con otros procesos activos supera la memoria disponible y hace que el sistema use swap en disco.

8. Análisis General

Los resultados demuestran algo que no siempre es obvio: en Python, la complejidad teórica es una guía necesaria pero no suficiente para predecir el rendimiento real. Hay tres factores que lo determinan de forma mucho más directa:

- El uso de BLAS a través de NumPy: el operador @ ejecuta código C compilado y optimizado para el hardware del equipo. Cualquier algoritmo que lo use correctamente será órdenes de magnitud más rápido que uno que opere elemento a elemento desde Python.
- El aprovechamiento de caché: trabajar con bloques pequeños que caben en caché L1 reduce los accesos a RAM y mejora el rendimiento de forma dramática, independientemente de la complejidad teórica.
- El overhead del intérprete: cada línea de Python tiene un costo fijo de ejecución. Loop unrolling, que en C reduce el tiempo al haber menos iteraciones, en Python lo aumenta porque hay más código a interpretar por iteración.

El caso de Strassen es el ejemplo más llamativo: tiene complejidad $O(n^{2.807})$ contra $O(n^3)$ del clásico, lo que debería hacerlo más rápido. En la práctica fue entre 6 y 7 veces más lento porque Python no puede amortizar el costo de la recursión y la gestión de memoria de la misma forma que un lenguaje compilado. En C o Fortran, Strassen sí supera al método clásico para matrices grandes.

9. Conclusiones

En Python, los algoritmos que mejor se desempeñan no son los de menor complejidad teórica sino los que mejor aprovechan las herramientas del lenguaje y del hardware. Los algoritmos por bloques con el operador @ de NumPy son entre 1.000 y 10.000 veces más rápidos que los iterativos puros, aunque su complejidad teórica sea idéntica $O(n^3)$.

El intento de correr con n=2048 confirma las limitaciones reales del enfoque iterativo y de Strassen en Python: no solo el tiempo de CPU se vuelve inmanejable, sino que Strassen excede la memoria disponible en equipos convencionales. Los algoritmos por bloques, en cambio, habrían corrido sin problema con 2048 o incluso tamaños mayores.

En cuanto al uso de RAM, los algoritmos más eficientes son los secuenciales por bloques, que trabajan con vistas de memoria sin crear copias adicionales. Strassen es el menos eficiente por su naturaleza recursiva que multiplica las matrices temporales en cada nivel.

Finalmente, el análisis de cómo se almacenan las columnas en formato row-major explica por qué el orden de acceso a memoria importa tanto: los algoritmos que acceden a columnas de forma no secuencial pagan un costo alto en fallos de caché, y los que estructuran su acceso por bloques neutralizan completamente ese problema.